

Evolving a Class's Implementation Details

When you design a class, carefully consider the class's interface before making that class available to other programmers. Ideally, you'll design the interface such that existing code will not break if you update the class's implementation details—that is, the internal data representation or how its method bodies are implemented.

If Python programmers follow convention and do not access attributes that begin with leading underscores, then class designers can evolve class implementation details without breaking client code.

Properties

It may seem that providing properties with both *setters* and *getters* has no benefit over accessing the data attributes directly, but there are subtle differences. A *getter* seems to allow clients to read the data at will, but the *getter* can control the formatting of the data. A *setter* can scrutinize attempts to modify the value of a data attribute to prevent the data from being set to an invalid value.

Utility Methods

Not all methods need to serve as part of a class's interface. Some serve as **utility methods** used only *inside* the class and are not intended to be part of the class's public interface used by client code. Such methods should be named with a single leading underscore. In other object-oriented languages like C++, Java and C#, such methods typically are implemented as private methods.

Module `datetime`

In professional Python development, rather than building your own classes to represent times and dates, you'll typically use the Python Standard Library's `datetime` module capabilities. For more details about the `datetime` module, see:

```
https://docs.python.org/3/library/datetime.html
```

10.5 SIMULATING “PRIVATE” ATTRIBUTES

In programming languages such as C++, Java and C#, classes state explicitly which class members are *publicly accessible*. Class members that may not be accessed outside a class definition are **private** and visible only within the class that defines them. Python programmers often use “private” attributes for data or utility methods that are essential to a class's inner workings but are not part of the class's public interface.

As you've seen, Python objects' attributes are *always* accessible. However, Python has a naming convention for “private” attributes. Suppose we want to create an object of class `Time` and to *prevent* the following assignment statement:

```
wake_up._hour = 100
```

that would set the hour to an invalid value. Rather than `_hour`, we can name the attribute `__hour` with *two* leading underscores. This convention indicates that `__hour` is “private” and should not be accessible to the class’s clients. To help prevent clients from accessing “private” attributes, Python *renames* them by preceding the attribute name with `_ClassName`, as in `_Time__hour`. This is called **name mangling**. If you try assign to `__hour`, as in

```
wake_up.__hour = 100
```

Python raises an `AttributeError`, indicating that the class does not have an `__hour` attribute. We’ll show this momentarily.

IPython Auto-Completion Shows Only “Public” Attributes

In addition, IPython does not show attributes with one or two leading underscores when you try to auto-complete an expression like

```
wake_up.
```

by pressing *Tab*. Only attributes that are part of the `wake_up` object’s “public” interface are displayed in the IPython auto-completion list.

Demonstrating “Private” Attributes

To demonstrate name mangling, consider class `PrivateClass` with one “public” data attribute `public_data` and one “private” data attribute `__private_data`:

[lick here to view code image](#)

```
1 # private.py
2 """Class with public and private  attributes."""
3
4 class PrivateClass:
5     """Class with public and private  attributes."""
6
7     def __init__(self):
8         """Initialize the public and private  attributes."""
9         self.public_data = "public" # public attribute
10        self.__private_data = "private" # private attribute
```

Let’s create an object of class `PrivateData` to demonstrate these data attributes:

[lick here to view code image](#)

```
In [1]: from private import PrivateClass
```

```
In [2]: my_object = PrivateClass()
```

Snippet [3] shows that we can access the `public_data` attribute directly:

[lick here to view code image](#)

```
In [3]: my_object.public_data
Out[3]: 'public'
```

However, when we attempt to access `__private_data` directly in snippet [4], we get an `AttributeError` indicating that the class does not have an attribute by that name:

[lick here to view code image](#)

```
In [4]: my_object.__private_data
-----
AttributeError                                Traceback (most recent call last)
<ipython-input-4-d896bdfd2053> in <module>()
----> 1 my_object.__private_data

AttributeError: 'PrivateClass' object has no attribute '__private_data'
```

This occurs because python changed the attribute's name. Unfortunately, the attribute `__private_data` is still indirectly accessible.

10.6 CASE STUDY: CARD SHUFFLING AND DEALING SIMULATION

Our next example presents two custom classes that you can use to shuffle and deal a deck of cards. Class `Card` represents a playing card that has a face ('Ace', '2', '3',, 'Jack', 'Queen', 'King') and a suit ('Hearts', 'Diamonds', 'Clubs', 'Spades'). Class `DeckOfCards` represents a deck of 52 playing cards as a list of `Card` objects. First, we'll test-drive these classes in an IPython session to demonstrate card shuffling and dealing capabilities and displaying the cards as text. Then we'll look at the class definitions. Finally, we'll use another IPython session to display the 52 cards as images using Matplotlib. We'll show you where to get nice-looking public-domain card images.

10.6.1 Test-Driving Classes `Card` and `DeckOfCards`

Before we look at classes `Card` and `DeckOfCards`, let's use an IPython session to demonstrate their capabilities.

Creating, Shuffling and Dealing the Cards

First, import class `DeckOfCards` from `deck.py` and create an object of the class:

[lick here to view code image](#)

```
In [1]: from deck import DeckOfCards
```

```
In [2]: deck_of_cards = DeckOfCards()
```

DeckOfCards method `__init__` creates the 52 Card objects in order by suit and by face within each suit. You can see this by printing the `deck_of_cards` object, which calls the DeckOfCards class's `__str__` method to get the deck's string representation. Read each row left-to-right to confirm that all the cards are displayed in order from each suit (Hearts, Diamonds, Clubs and Spades):

[lick here to view code image](#)

```
In [3]: print(deck_of_cards)
Ace of Hearts      2 of Hearts      3 of Hearts      4 of Hearts
5 of Hearts       6 of Hearts      7 of Hearts      8 of Hearts
9 of Hearts       10 of Hearts     Jack of Hearts   Queen of Hearts
King of Hearts    Ace of Diamonds  2 of Diamonds   3 of Diamonds
4 of Diamonds    5 of Diamonds   6 of Diamonds   7 of Diamonds
8 of Diamonds    9 of Diamonds   10 of Diamonds  Jack of Diamonds
Queen of Diamonds King of Diamonds Ace of Clubs     2 of Clubs
3 of Clubs       4 of Clubs      5 of Clubs      6 of Clubs
7 of Clubs       8 of Clubs      9 of Clubs      10 of Clubs
Jack of Clubs    Queen of Clubs   King of Clubs    Ace of Spades
2 of Spades     3 of Spades     4 of Spades     5 of Spades
6 of Spades     7 of Spades     8 of Spades     9 of Spades
10 of Spades    Jack of Spades   Queen of Spades  King of Spades
```

Next, let's shuffle the deck and print the `deck_of_cards` object again. We did not specify a seed for reproducibility, so each time you shuffle, you'll get different results:

[lick here to view code image](#)

```
In [4]: deck_of_cards.shuffle()

In [5]: print(deck_of_cards)
King of Hearts    Queen of Clubs    Queen of Diamonds 10 of Clubs
5 of Hearts       7 of Hearts      4 of Hearts       2 of Hearts
5 of Clubs        8 of Diamonds    3 of Hearts       10 of Hearts
8 of Spades       5 of Spades      Queen of Spades   Ace of Clubs
8 of Clubs        7 of Spades      Jack of Diamonds  10 of Spades
4 of Diamonds     8 of Hearts      6 of Spades       King of Spades
9 of Hearts       4 of Spades      6 of Clubs        King of Clubs
3 of Spades       9 of Diamonds    3 of Clubs        Ace of Spades
Ace of Hearts     3 of Diamonds    2 of Diamonds     6 of Hearts
King of Diamonds  Jack of Spades   Jack of Clubs     2 of Spades
5 of Diamonds     4 of Clubs       Queen of Hearts   9 of Clubs
10 of Diamonds    2 of Clubs       Ace of Diamonds   7 of Diamonds
9 of Spades       Jack of Hearts    6 of Diamonds     7 of Clubs
```

Dealing Cards

We can deal one `Card` at a time by calling method `deal_card`. IPython calls the returned `Card` object's `__repr__` method to produce the string output shown in the `Out[]` prompt:

[lick here to view code image](#)

```
In [6]: deck_of_cards.deal_card()
Out[6]: Card(face='King', suit='Hearts')
```

Class `Card`'s Other Features

To demonstrate class `Card`'s `__str__` method, let's deal another card and pass it to the built-in `str` function:

[lick here to view code image](#)

```
In [7]: card = deck_of_cards.deal_card()

In [8]: str(card)
Out[8]: 'Queen of Clubs'
```

Each `Card` has a corresponding image file name, which you can get via the `image_name` read-only property. We'll use this soon when we display the `Cards` as images:

[lick here to view code image](#)

```
In [9]: card.image_name
Out[9]: 'Queen_of_Clubs.png'
```

10.6.2 Class `Card`—Introducing Class Attributes

Each `Card` object contains three string properties representing that `Card`'s face, suit and `image_name` (a file name containing a corresponding image). As you saw in the preceding section's IPython session, class `Card` also provides methods for initializing a `Card` and for getting various string representations.

Class Attributes `FACES` and `SUITS`

Each object of a class has its own copies of the class's data attributes. For example, each `Account` object has its own `name` and `balance`. Sometimes, an attribute should be shared by *all* objects of a class. A **class attribute** (also called a **class variable**) represents *class-wide* information. It belongs to the *class*, not to a specific object of that class. Class `Card` defines two class attributes (lines 5–7):

- `FACES` is a list of the card face names.
- `SUITS` is a list of the card suit names.

[lick here to view code image](#)

```
1 # card.py
2 """Card class that represents a playing card and its image file name."""
3
4 class Card:
5     FACES = ['Ace', '2', '3', '4', '5', '6',
6             '7', '8', '9', '10', 'Jack', 'Queen', 'King']
7     SUITS = ['Hearts', 'Diamonds', 'Clubs', 'Spades']
8
```

You define a class attribute by assigning a value to it inside the class's definition, but not inside any of the class's methods or properties (in which case, they'd be local variables). `FACES` and `SUITS` are *constants* that are not meant to be modified. Recall that the *Style Guide for Python Code* recommends naming your constants with all capital letters.⁴

⁴ Recall that Python does not have true constants, so `FACES` and `SUITS` are still modifiable.

We'll use elements of these lists to initialize each `Card` we create. However, we do not need a separate copy of each list in every `Card` object. Class attributes can be accessed through any object of the class, but are typically accessed through the class's name (as in, `Card.FACES` or `Card.SUITS`). Class attributes exist as soon as you import their class's definition.

Card Method `__init__`

When you create a `Card` object, method `__init__` defines the object's `_face` and `_suit` data attributes:

[lick here to view code image](#)

```
9     def __init__(self, face, suit):
10         """Initialize a Card with a face and suit."""
11         self._face = face
12         self._suit = suit
13
```

Read-Only Properties `face`, `suit` and `image_name`

Once a `Card` is created, its `face`, `suit` and `image_name` do not change, so we implement these as read-only properties (lines 14–17, 19–22 and 24–27). Properties `face` and `suit` return the corresponding data attributes `_face` and `_suit`. A property is not required to have a corresponding data attribute. To demonstrate this, the `Card` property `image_name`'s value is *created dynamically* by getting the `Card` object's string representation with `str(self)`, replacing any spaces with underscores and appending the `'.png'` filename extension. So, 'Ace of Spades' becomes 'Ace_of_Spades.png'. We'll use this file name to load a PNG-format image representing the `Card`. PNG (Portable Network Graphics) is a popular image format for web-based images.

[lick here to view code image](#)

```
14     @property
15     def face(self):
16         """Return the Card's self._face    value."""
17         return self._face
18
19     @property
20     def suit(self):
21         """Return the Card's self._suit    value."""
22         return self._suit
23
24     @property
25     def image_name(self):
26         """Return the Card's image file    name."""
27         return str(self).replace(' ', '_') + '.png'
28
```

Methods That Return String Representations of a Card

Class `Card` provides three special methods that return string representations. As in class `Time`, method `__repr__` returns a string representation that looks like a constructor expression for creating and initializing a `Card` object:

[lick here to view code image](#)

```
29     def __repr__(self):
30         """Return string representation for    repr()."""
31         return f"Card(face='{self.face}', suit='{self.suit}')"
32
```

Method `__str__` returns a string of the format '*face* of *suit*', such as 'Ace of Hearts':

[lick here to view code image](#)

```
33     def __str__(self):
34         """Return string representation for    str()."""
35         return f'{self.face} of {self.suit}'
36
```

When the preceding section's IPython session printed the entire deck, you saw that the `Cards` were displayed in four left-aligned columns. As you'll see in the `__str__` method of class `DeckOfCards`, we use f-strings to format the `Cards` in fields of 19 characters each. Class `Card`'s special method `__format__` is called when a `Card` object is *formatted* as a string, such as in an f-string:

[lick here to view code image](#)

```
37     def __format__(self, format):
38         """Return formatted string    representation for str()."""
```

This method's second argument is the format string used to format the object. To use the `format` parameter's value as the format specifier, enclose the parameter name in braces to the *right* of the colon. In this case, we're formatting the `Card` object's string representation returned by `str(self)`. We'll discuss `__format__` again when we present the `__str__` method in class `DeckOfCards`.

10.6.3 Class `DeckOfCards`

Class `DeckOfCards` has a class attribute `NUMBER_OF_CARDS`, representing the number of Cards in a deck, and creates two data attributes:

- `_current_card` keeps track of which `Card` will be dealt next (0–51) and
- `_deck` (line 12) is a list of 52 `Card` objects.

Method `__init__`

`DeckOfCards` method `__init__` initializes a `_deck` of Cards. The `for` statement fills the list `_deck` by appending new `Card` objects, each initialized with two strings—one from the list `Card.FACES` and one from `Card.SUITS`. The calculation `count % 13` *always* results in a value from 0 to 12 (the 13 indices of `Card.FACES`), and the calculation `count // 13` *always* results in a value from 0 to 3 (the four indices of `Card.SUITS`). When the `_deck` list is initialized, it contains the Cards with faces 'Ace' through 'King' in order for all the Hearts, then the Diamonds, then the Clubs, then the Spades.

[lick here to view code image](#)

```

1 # deck.py
2 """Deck class represents a deck of Cards."""
3 import random
4 from card import Card
5
6 class DeckOfCards:
7     NUMBER_OF_CARDS = 52 # constant number of Cards
8
9     def __init__(self):
10         """Initialize the deck."""
11         self._current_card = 0
12         self._deck = []
13
14         for count in range(DeckOfCards.NUMBER_OF_CARDS):
15             self._deck.append(Card(Card.FACES[count % 13],
16                                   Card.SUITS[count // 13]))
17 
```

Method `shuffle`

Method `shuffle` resets `_current_card` to 0, then shuffles the Cards in `_deck` using the

random module's shuffle function:

[lick here to view code image](#)

```
18     def shuffle(self):
19         """Shuffle deck."""
20         self._current_card = 0
21         random.shuffle(self._deck)
22
```

Method `deal_card`

Method `deal_card` deals one Card from `_deck`. Recall that `_current_card` indicates the index (0–51) of the next Card to be dealt (that is, the Card at the top of the deck). Line 26 tries to get the `_deck` element at index `_current_card`. If successful, the method increments `_current_card` by 1, then returns the Card being dealt; otherwise, the method returns `None` to indicate there are no more Cards to deal.

[lick here to view code image](#)

```
23     def deal_card(self):
24         """Return one Card."""
25         try:
26             card = self._deck[self._current_card]
27             self._current_card += 1
28             return card
29         except:
30             return None
31
```

Method `__str__`

Class `DeckOfCards` also defines special method `__str__` to get a string representation of the deck in four columns with each Card left aligned in a field of 19 characters. When line 37 formats a given Card, its `__format__` special method is called with format specifier '`<19`' as the method's `format` argument. Method `__format__` then uses '`<19`' to create the Card's formatted string representation.

[lick here to view code image](#)

```
32     def __str__(self):
33         """Return a string representation of the current _deck."""
34         s = ''
35
36         for index, card in enumerate(self._deck):
37             s += f'{self._deck[index]:<19}'
38             if (index + 1) % 4 == 0:
39                 s += '\n'
40
41         return s
```

10.6.4 Displaying Card Images with Matplotlib

So far, we've displayed Cards as text. Now, let's display Card images. For this demonstration, we downloaded public-domain ⁵ card images from Wikimedia Commons:

⁵ <https://creativecommons.org/publicdomain/zero/1.0/deed.en>.

https://commons.wikimedia.org/wiki/-Category:SVG_English_pattern_playing_cards

These are located in the `ch10` examples folder's `card_images` subfolder. First, let's create a `DeckOfCards`:

[lick here to view code image](#)

```
In [1]: from deck import DeckOfCards

In [2]: deck_of_cards = DeckOfCards()
```

Enable Matplotlib in IPython

Next, enable Matplotlib support in IPython by using the `%matplotlib` magic:

[lick here to view code image](#)

```
In [3]: %matplotlib
Using matplotlib backend: Qt5Agg
```

Create the Base Path for Each Image

Before displaying each image, we must load it from the `card_images` folder. We'll use the `pathlib` module's **Path class** to construct the full path to each image on our system.

Snippet [5] creates a `Path` object for the current folder (the `ch10` examples folder), which is represented by `'.'`, then uses `Path` method **`joinpath`** to append the subfolder containing the card images:

[lick here to view code image](#)

```
In [4]: from pathlib import Path

In [5]: path = Path('.').joinpath('card_images')
```

Import the Matplotlib Features

Next, let's import the Matplotlib modules we'll need to display the images. We'll use a function from **`matplotlib.image`** to load the images:

[lick here to view code image](#)

```
In [6]: import matplotlib.pyplot as plt

In [7]: import matplotlib.image as mpimg
```

Create the Figure and Axes Objects

The following snippet uses Matplotlib function **subplots** to create a Figure object in which we'll display the images as 52 *subplots* with four rows (**nrows**) and 13 columns (**ncols**). The function returns a tuple containing the Figure and an array of the subplots' Axes objects. We unpack these into variables **figure** and **axes_list**:

[lick here to view code image](#)

```
In [8]: figure, axes_list = plt.subplots(nrows=4, ncols=13)
```

When you execute this statement in IPython, the Matplotlib window appears immediately with 52 empty subplots.

Configure the Axes Objects and Display the Images

Next, we iterate through all the Axes objects in **axes_list**. Recall that **ravel** provides a one-dimensional view of a multidimensional array. For each Axes object, we perform the following tasks:

- We're not plotting data, so we do not need axis lines and labels for each image. The first two statements in the loop hide the *x*- and *y*-axes.
- The third statement deals a Card and gets its **image_name**.
- The fourth statement uses Path method **joinpath** to append the **image_name** to the Path, then calls Path method **resolve** to determine the full path to the image on our system. We pass the resulting Path object to the built-in **str** function to get the string representation of the image's location. Then, we pass that string to the **matplotlib.image** module's **imread function**, which loads the image.
- The last statement calls Axes method **imshow** to display the current image in the current subplot.

[lick here to view code image](#)

```
In [9]: for axes in axes_list.ravel():
...:     axes.get_xaxis().set_visible(False)
...:     axes.get_yaxis().set_visible(False)
...:     image_name = deck_of_cards.deal_card().image_name
...:     img = mpimg.imread(str(path.joinpath(image_name).resolve()))
```

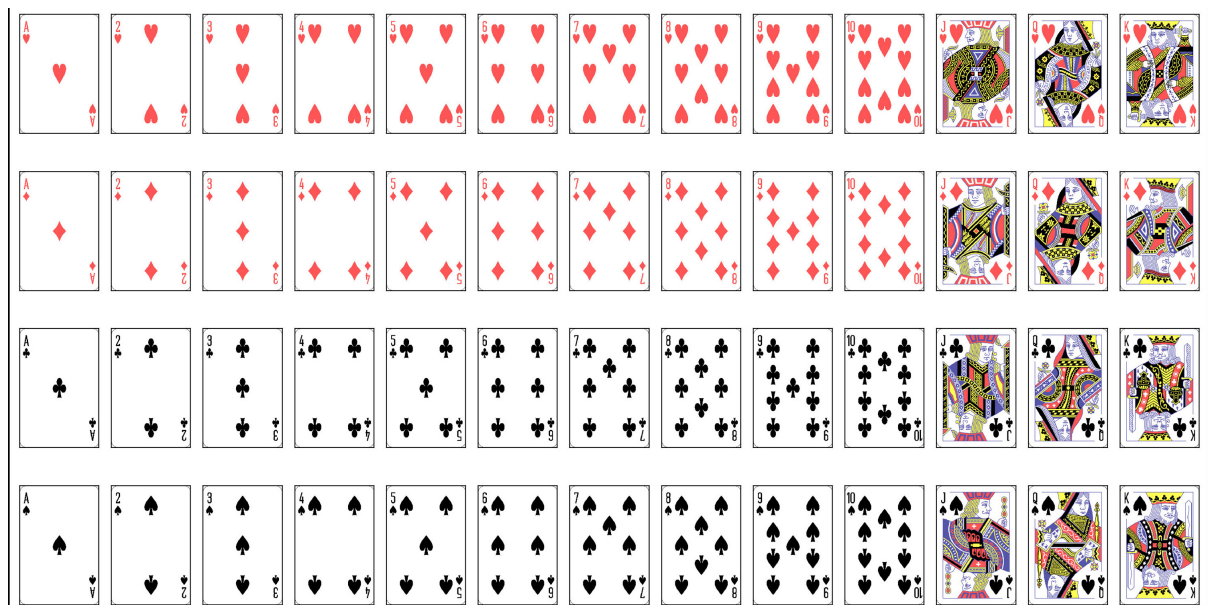
```
...:     axes.imshow(img)
...:
```

Maximize the Image Sizes

At this point, all the images are displayed. To make the cards as large as possible, you can maximize the window, then call the Matplotlib Figure object's **`tight_layout` method**. This removes most of the extra white space in the window:

```
In [10]: figure.tight_layout()
```

The following image shows the contents of the resulting window:



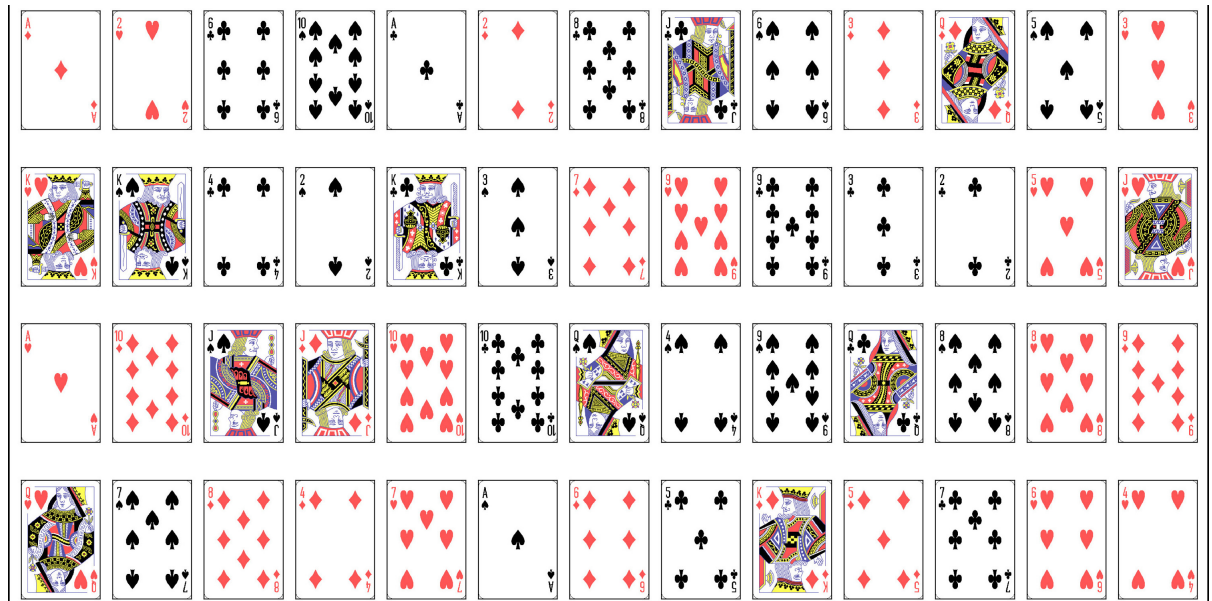
Shuffle and Re-Deal the Deck

To see the images shuffled, call method `shuffle`, then re-execute snippet [9]'s code:

[lick here to view code image](#)

```
In [11]: deck_of_cards.shuffle()

In [12]: for axes in axes_list.ravel():
...:     axes.get_xaxis().set_visible(False)
...:     axes.get_yaxis().set_visible(False)
...:     image_name = deck_of_cards.deal_card().image_name
...:     img = mpimg.imread(str(path.joinpath(image_name).resolve()))
...:     axes.imshow(img)
...:
```



10.7 INHERITANCE: BASE CLASSES AND SUBCLASSES

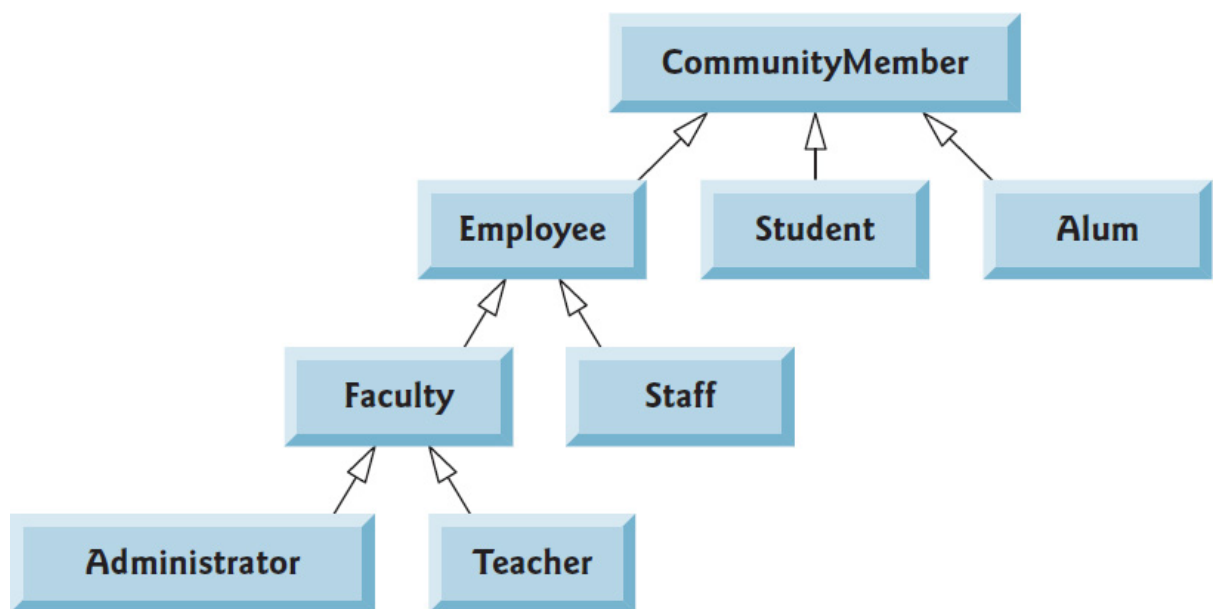
Often, an object of one class *is an* object of another class as well. For example, a `CarLoan` *is a* `Loan` as are `HomeImprovementLoans` and `MortgageLoans`. Class `CarLoan` can be said to inherit from class `Loan`. In this context, class `Loan` is a base class, and class `CarLoan` is a subclass. A `CarLoan` *is a* specific type of `Loan`, but it's incorrect to claim that every `Loan` *is a* `CarLoan`—the `Loan` could be of any type. The following table lists simple examples of base classes and subclasses—base classes tend to be “more general” and subclasses “more specific”:

Base class	Subclasses
Student	GraduateStudent, UndergraduateStudent
Shape	Circle, Triangle, Rectangle, Sphere, Cube
Loan	CarLoan, HomeImprovementLoan, MortgageLoan
Employee	Faculty, Staff
BankAccount	CheckingAccount, SavingsAccount

ecause every subclass object *is an* object of its base class, and one base class can have many subclasses, the set of objects represented by a base class is often larger than the set of objects represented by any of its subclasses. For example, the base class `Vehicle` represents *all* vehicles, including cars, trucks, boats, bicycles and so on. By contrast, subclass `Car` represents a smaller, more specific subset of vehicles.

CommunityMember Inheritance Hierarchy

Inheritance relationships form tree-like *hierarchical* structures. A base class exists in a hierarchical relationship with its subclasses. Let’s develop a sample class hierarchy (shown in the following diagram), also called an **inheritance hierarchy**. A university community has thousands of members, including employees, students and alumni. Employees are either faculty or staff members. Faculty members are either administrators (e.g., deans and department chairpersons) or teachers. The hierarchy could contain many other classes. For example, students can be graduate or undergraduate students. Undergraduate students can be freshmen, sophomores, juniors or seniors. With **single inheritance**, a class is derived from *one* base class. With **multiple inheritance**, a subclass inherits from *two or more* base classes. Single inheritance is straightforward. Multiple inheritance is beyond the scope of this book. Before you use it, search online for the “diamond problem in Python multiple inheritance.”

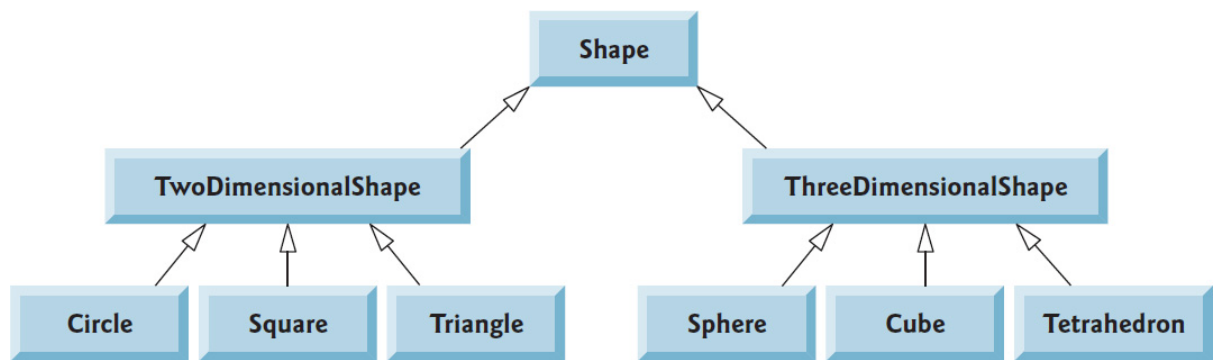


Each arrow in the hierarchy represents an *is-a* relationship. As we follow the arrows upward in this class hierarchy, we can state, for example, that “an `Employee` *is a* `CommunityMember`” and “a `Teacher` *is a* `Faculty` member.” `CommunityMember` is the direct base class of `Employee`, `Student` and `Alum` and is an indirect base class of all the other classes in the diagram. Starting from the bottom, you can follow the arrows and apply the *is-a* relationship up to the topmost superclass. For example, `Administrator` *is a* `Faculty` member, *is an* `Employee`, *is a* `Community-Member` and, of course, ultimately *is an* object.

Shape Inheritance Hierarchy

Now consider the `Shape` inheritance hierarchy in the following class diagram, which begins

ith base class `Shape`, followed by subclasses `TwoDimensionalShape` and `ThreeDimensionalShape`. Each `Shape` is either a `TwoDimensionalShape` or a `ThreeDimensionalShape`. The third level of this hierarchy contains *specific* types of `TwoDimensionalShapes` and `ThreeDimensionalShapes`. Again, we can follow the arrows from the bottom of the diagram to the topmost base class in this class hierarchy to identify several *is-a* relationships. For example, a `Triangle` *is a* `TwoDimensionalShape` and *is a* `Shape`, while a `Sphere` *is a* `ThreeDimensionalShape` and *is a* `Shape`. This hierarchy could contain many other classes. For example, ellipses and trapezoids also are `TwoDimensionalShapes`, and cones and cylinders also are `ThreeDimensionalShapes`.



“is a” vs. “has a”

Inheritance produces **“is-a” relationships** in which an object of a subclass type may also be treated as an object of the base-class type. You’ve also seen “has-a” (composition) relationships in which a class has references to one or more objects of other classes as members.

10.8 BUILDING AN INHERITANCE HIERARCHY; INTRODUCING POLYMORPHISM

Let’s use a hierarchy containing types of employees in a company’s payroll app to discuss the relationship between a base class and its subclass. All employees of the company have a lot in common, but *commission employees* (who will be represented as objects of a base class) are paid a percentage of their sales, while *salaried commission employees* (who will be represented as objects of a subclass) receive a percentage of their sales *plus* a base salary.

First, we present *base class* `CommissionEmployee`. Next, we create a *subclass* `SalariedCommissionEmployee` that inherits from class `CommissionEmployee`. Then, we use an IPython session to create a `SalariedCommissionEmployee` object and demonstrate that it has all the capabilities of the base class *and* the subclass, but calculates its earnings differently.

10.8.1 Base Class `CommissionEmployee`

Consider class `CommissionEmployee`, which provides the following features:

- Method `__init__` (lines 8–15), which creates the data attributes `_first_name`,

`_last_name` and `_ssn` (Social Security number), and uses the setters of properties `gross_sales` and `commission_rate` to create their corresponding data attributes.

- Read-only properties `first_name` (lines 17–19), `last_name` (lines 21–23) and `ssn` (line 25–27), which return the corresponding data attributes.
- Read-write properties `gross_sales` (lines 29–39) and `commission_rate` (lines 41–52), in which the setters perform data validation.
- Method `earnings` (lines 54–56), which calculates and returns a `CommissionEmployee`'s earnings.
- Method `__repr__` (lines 58–64), which returns a string representation of a `CommissionEmployee`.

[lick here to view code image](#)

```
1 # commissionemployee.py
2 """CommissionEmployee base class."""
3 from decimal import Decimal
4
5 class CommissionEmployee:
6     """An employee who gets paid commission based on gross sales."""
7
8     def __init__(self, first_name, last_name, ssn,
9                 gross_sales, commission_rate):
10         """Initialize CommissionEmployee's attributes."""
11         self._first_name = first_name
12         self._last_name = last_name
13         self._ssn = ssn
14         self.gross_sales = gross_sales # validate via property
15         self.commission_rate = commission_rate # validate via property
16
17     @property
18     def first_name(self):
19         return self._first_name
20
21     @property
22     def last_name(self):
23         return self._last_name
24
25     @property
26     def ssn(self):
27         return self._ssn
28
29     @property
30     def gross_sales(self):
31         return self._gross_sales
32
33     @gross_sales.setter
34     def gross_sales(self, sales):
35         """Set gross sales or raise ValueError if invalid."""
36         if sales < Decimal('0.00'):
37             raise ValueError('Gross sales must be >= to 0')
38
```

```

39         self._gross_sales = sales
40
41     @property
42     def commission_rate(self):
43         return self._commission_rate
44
45     @commission_rate.setter
46     def commission_rate(self, rate):
47         """Set commission rate or raise ValueError if invalid."""
48         if not (Decimal('0.0') < rate < Decimal('1.0')):
49             raise ValueError(
50                 'Interest rate must be greater than 0 and less than 1')
51
52         self._commission_rate = rate
53
54     def earnings(self):
55         """Calculate earnings."""
56         return self.gross_sales * self.commission_rate
57
58     def __repr__(self):
59         """Return string representation for repr()."""
60         return ('CommissionEmployee: ' +
61                 f'{self.first_name} {self.last_name}\n' +
62                 f'social security number: {self.ssn}\n' +
63                 f'gross sales: {self.gross_sales:.2f}\n' +
64                 f'commission rate: {self.commission_rate:.2f}')

```

Properties `first_name`, `last_name` and `ssn` are read-only. We chose not to validate them, though we could have. For example, we could validate the first and last names—perhaps by ensuring that they’re of a reasonable length. We could validate the Social Security number to ensure that it contains nine digits, with or without dashes (for example, to ensure that it’s in the format `###-##-####` or `#####`, where each `#` is a digit).

All Classes Inherit Directly or Indirectly from Class `object`

You use inheritance to create new classes from existing ones. In fact, *every* Python class inherits from an existing class. When you do not explicitly specify the base class for a new class, Python assumes that the class inherits directly from class `object`. The Python class hierarchy begins with class `object`, the direct or indirect base class of *every* class. So, class `CommissionEmployee`’s header could have been written as

```
class CommissionEmployee(object):
```

The parentheses after `CommissionEmployee` indicate inheritance and may contain a single class for single inheritance or a comma-separated list of base classes for multiple inheritance. Once again, multiple inheritance is beyond the scope of this book.

Class `CommissionEmployee` inherits all the methods of class `object`. Class `object` does not have any data attributes. Two of the many methods inherited from `object` are `__repr__` and `__str__`. So *every* class has these methods that return string representations of the objects on which they’re called. When a base-class method

implementation is inappropriate for a derived class, that method can be **overridden** (i.e., redefined) in the derived class with an appropriate implementation. Method `__repr__` (lines 58–64) overrides the default implementation inherited into class `CommissionEmployee` from class `object`.⁶

⁶ ee <https://docs.python.org/3/reference/datamodel.html> for objects overridable methods.

Testing Class `CommissionEmployee`

Let’s quickly test some of `CommissionEmployee`’s features. First, create and display a `CommissionEmployee`:

[lick here to view code image](#)

```
In [1]: from commissionemployee import CommissionEmployee

In [2]: from decimal import Decimal

In [3]: c = CommissionEmployee('Sue', 'Jones', '333-33-3333',
...:      Decimal('10000.00'), Decimal('0.06'))
...:

In [4]: c
Out[4]:
CommissionEmployee: Sue Jones
social security number: 333-33-3333
gross sales: 10000.00
commission rate: 0.06
```

Next, let’s calculate and display the `CommissionEmployee`’s earnings:

[lick here to view code image](#)

```
In [5]: print(f'{c.earnings():.2f}')
600.00
```

Finally, let’s change the `CommissionEmployee`’s gross sales and commission rate, then recalculate the earnings:

[lick here to view code image](#)

```
In [6]: c.gross_sales = Decimal('20000.00')

In [7]: c.commission_rate = Decimal('0.1')

In [8]: print(f'{c.earnings():.2f}')
2,000.00
```

10.8.2 Subclass `SalariedCommissionEmployee`

With single inheritance, the subclass starts essentially the same as the base class. The real strength of inheritance comes from the ability to define in the subclass additions, replacements or refinements for the features inherited from the base class.

Many of a `SalariedCommissionEmployee`'s capabilities are similar, if not identical, to those of class `CommissionEmployee`. Both types of employees have first name, last name, Social Security number, gross sales and commission rate data attributes, and properties and methods to manipulate that data. To create class `SalariedCommissionEmployee` *without* using inheritance, we could have *copied* class `CommissionEmployee`'s code and *pasted* it into class `SalariedCommissionEmployee`. Then we could have modified the new class to include a base salary data attribute, and the properties and methods that manipulate the base salary, including a new `earnings` method. This *copy-and-paste approach* is often error-prone. Worse yet, it can spread many physical copies of the same code (including errors) throughout a system, making your code less maintainable. Inheritance enables us to “absorb” the features of a class *without* duplicating code. Let's see how.

Declaring Class `SalariedCommissionEmployee`

We now declare the subclass `SalariedCommissionEmployee`, which *inherits* most of its capabilities from class `CommissionEmployee` (line 6). A `SalariedCommissionEmployee` *is a* `CommissionEmployee` (because inheritance passes on the capabilities of class `CommissionEmployee`), but class `SalariedCommissionEmployee` also has the following features:

- Method `__init__` (lines 10–15), which initializes all the data inherited from class `CommissionEmployee` (we'll say more about this momentarily), then uses the `base_salary` property's setter to create a `_base_salary` data attribute.
- Read-write property `base_salary` (lines 17–27), in which the setter performs data validation.
- A customized version of method `earnings` (lines 29–31).
- A customized version of method `__repr__` (lines 33–36).

[lick here to view code image](#)

```
1 # salariedcommissionemployee.py
2 """SalariedCommissionEmployee derived from CommissionEmployee."""
3 from commissionemployee import CommissionEmployee
4 from decimal import Decimal
5
6 class SalariedCommissionEmployee(CommissionEmployee):
7     """An employee who gets paid a salary plus
8     commission based on gross sales."""
9
```

```

10     def __init__(self, first_name, last_name, ssn,
11                  gross_sales, commission_rate, base_salary):
12         """Initialize SalariedCommissionEmployee's attributes."""
13         super().__init__(first_name, last_name, ssn,
14                          gross_sales, commission_rate)
15         self.base_salary = base_salary # validate via property
16
17     @property
18     def base_salary(self):
19         return self._base_salary
20
21     @base_salary.setter
22     def base_salary(self, salary):
23         """Set base salary or raise ValueError if invalid."""
24         if salary < Decimal('0.00'):
25             raise ValueError('Base salary must be >= to 0')
26
27         self._base_salary = salary
28
29     def earnings(self):
30         """Calculate earnings."""
31         return super().earnings() + self.base_salary
32
33     def __repr__(self):
34         """Return string representation for repr()."""
35         return ('Salaried' + super().__repr__() +
36                f'\nbase salary: {self.base_salary:.2f}')

```

Inheriting from Class `CommissionEmployee`

To inherit from a class, you must first import its definition (line 3). Line 6

```
class SalariedCommissionEmployee(CommissionEmployee):
```

specifies that class `SalariedCommissionEmployee` *inherits* from `CommissionEmployee`. Though you do not see class `CommissionEmployee`'s data attributes, properties and methods in class `SalariedCommissionEmployee`, they're nevertheless part of the new class, as you'll soon see.

Method `__init__` and Built-In Function `super`

Each subclass `__init__` must explicitly call its base class's `__init__` to initialize the data attributes inherited from the base class. This call should be the first statement in the subclass's `__init__` method. `SalariedCommissionEmployee`'s `__init__` method explicitly calls class `CommissionEmployee`'s `__init__` method (lines 13–14) to initialize the base-class portion of a `SalariedCommissionEmployee` object (that is, the five inherited data attributes from class `CommissionEmployee`). The notation `super().__init__` uses the built-in function **super** to locate and call the base class's `__init__` method, passing the five arguments that initialize the inherited data attributes.

Overriding Method `earnings`

Class `SalariedCommissionEmployee`'s `earnings` method (lines 29–31) overrides class `CommissionEmployee`'s `earnings` method (section 10.8.1, lines 54–56) to calculate the earnings of a `SalariedCommissionEmployee`. The new version obtains the portion of the earnings based on *commission alone* by calling `CommissionEmployee`'s `earnings` method with the expression `super().earnings()` (line 31). `SalariedCommissionEmployee`'s `earnings` method then adds the `base_salary` to this value to calculate the total earnings. By having `SalariedCommissionEmployee`'s `earnings` method invoke `CommissionEmployee`'s `earnings` method to calculate part of a `SalariedCommissionEmployee`'s earnings, we avoid duplicating the code and reduce code-maintenance problems.

Overriding Method `__repr__`

`SalariedCommissionEmployee`'s `__repr__` method (lines 33–36) overrides class `CommissionEmployee`'s `__repr__` method (section 10.8.1, lines 58–64) to return a `String` representation that's appropriate for a `SalariedCommissionEmployee`. The subclass creates part of the string representation by concatenating `'Salaried'` and the string returned by `super().__repr__()`, which calls `CommissionEmployee`'s `__repr__` method. The overridden method then concatenates the base salary information and returns the resulting string.

Testing Class `SalariedCommissionEmployee`

Let's test class `SalariedCommissionEmployee` to show that it indeed inherited capabilities from class `CommissionEmployee`. First, let's create a `SalariedCommissionEmployee` and print all of its properties:

[lick here to view code image](#)

```
In [9]: from salariedcommissionemployee import SalariedCommissionEmployee

In [10]: s = SalariedCommissionEmployee('Bob', 'Lewis', '444-44-4444',
...:      Decimal('5000.00'), Decimal('0.04'), Decimal('300.00'))
...:

In [11]: print(s.first_name, s.last_name, s.ssn, s.gross_sales,
...:      s.commission_rate, s.base_salary)
Bob Lewis 444-44-4444 5000.00 0.04 300.00
```

Notice that the `SalariedCommissionEmployee` object has *all* of the properties of classes `CommissionEmployee` *and* `SalariedCommissionEmployee`.

Next, let's calculate and display the `SalariedCommissionEmployee`'s earnings. Because we call method `earnings` on a `SalariedCommissionEmployee` object, the *subclass version* of the method executes:

[lick here to view code image](#)

```
In [12]: print(f'{s.earnings():,.2f}')
500.00
```

Now, let's modify the `gross_sales`, `commission_rate` and `base_salary` properties, then display the updated data via the `Salaried-Commission-Employee`'s `__repr__` method:

[lick here to view code image](#)

```
In [13]: s.gross_sales = Decimal('10000.00')

In [14]: s.commission_rate = Decimal('0.05')

In [15]: s.base_salary = Decimal('1000.00')

In [16]: print(s)
SalariedCommissionEmployee: Bob Lewis
social security number: 444-44-4444
gross sales: 10000.00
commission rate: 0.05
base salary: 1000.00
```

Again, because this method is called on a `SalariedCommissionEmployee` object, the *subclass version* of the method executes. Finally, let's calculate and display the `Salaried-Commission-Employee`'s updated earnings:

[lick here to view code image](#)

```
In [17]: print(f'{s.earnings():,.2f}')
1,500.00
```

Testing the “is a” Relationship

Python provides two built-in functions—`issubclass` and `isinstance`—for testing “is a” relationships. Function `issubclass` determines whether one class is derived from another:

[lick here to view code image](#)

```
In [18]: issubclass(SalariedCommissionEmployee, CommissionEmployee)
Out[18]: True
```

Function `isinstance` determines whether an object has an “is a” relationship with a specific type. Because `SalariedCommissionEmployee` inherits from `CommissionEmployee`, both of the following snippets return `True`, confirming the “is a” relationship

[lick here to view code image](#)

```
In [19]: isinstance(s, CommissionEmployee)
```

```
Out[19]: True

In [20]: isinstance(s, SalariedCommissionEmployee)
Out[20]: True
```

10.8.3 Processing `CommissionEmployee` and `SalariedCommissionEmployee` Polymorphically

With inheritance, every object of a subclass also may be treated as an object of that subclass's base class. We can take advantage of this “subclass-object-is-a-base-class-object” relationship to perform some interesting manipulations. For example, we can place objects related through inheritance into a list, then iterate through the list and treat each element as a base-class object. This allows a variety of objects to be processed in a *general* way. Let's demonstrate this by placing the `CommissionEmployee` and `SalariedCommissionEmployee` objects in a list, then for each element displaying its string representation and earnings:

[lick here to view code image](#)

```
In [21]: employees = [c, s]

In [22]: for employee in employees:
...:     print(employee)
...:     print(f'{employee.earnings():, .2f}\n')
...:
CommissionEmployee: Sue Jones
social security number: 333-33-3333
gross sales: 20000.00
commission rate: 0.10
2,000.00

SalariedCommissionEmployee: Bob Lewis
social security number: 444-44-4444
gross sales: 10000.00
commission rate: 0.05
base salary: 1000.00
1,500.00
```

As you can see, the correct string representation and earnings are displayed for each employee. This is called *polymorphism*—a key capability of object-oriented programming (OOP).

10.8.4 A Note About Object-Based and Object-Oriented Programming

Inheritance with method overriding is a powerful way to build software components that are *like* existing components but need to be customized to your application's unique needs. In the Python open-source world, there are a huge number of well-developed class libraries for which your programming style is:

- know what libraries are available,

- know what classes are available,
- make objects of existing classes, and
- send them messages (that is, call their methods).

This style of programming is called *object-based programming (OBP)*. When you do composition with objects of known classes, you're still doing object-based programming. Adding inheritance with overriding to customize methods to the unique needs of your applications and possibly process objects polymorphically is called *object-oriented programming (OOP)*. If you do composition with objects of inherited classes, that's also object-oriented programming.

10.9 DUCK TYPING AND POLYMORPHISM

Most other object-oriented programming languages require inheritance-based “is a” relationships to achieve polymorphic behavior. Python is more flexible. It uses a concept called duck typing, which the Python documentation describes as:

*A programming style which does not look at an object's type to determine if it has the right interface; instead, the method or attribute is simply called or used (“If it looks like a duck and quacks like a duck, it must be a duck.”).*⁷

⁷ <https://docs.python.org/3/glossary.html#term-duck-typing>.

So, when processing an object at execution time, its type does not matter. As long as the object has the data attribute, property or method (with the appropriate parameters) you wish to access, the code will work.

Let's reconsider the loop at the end of section 10.8.3, which processes a list of employees:

```
for employee in employees:
    print(employee)
    print(f'{employee.earnings():.2f}\n')
```

In Python, this loop works properly as long as `employees` contains only objects that:

- can be displayed with `print` (that is, they have a string representation) and
- have an `earnings` method which can be called with no arguments.

All classes inherit from `object` directly or indirectly, so they *all* inherit the default methods for obtaining string representations that `print` can display. If a class has an `earnings` method that can be called with no arguments, we can include objects of that class in the list `employees`, even if the object's class does not have an “is a” relationship with class `CommissionEmployee`. To demonstrate this, consider class `WellPaidDuck`:

[lick here to view code image](#)

```
In [1]: class WellPaidDuck:
...:     def __repr__(self):
...:         return 'I am a well-paid duck'
...:     def earnings(self):
...:         return Decimal('1_000_000.00')
...:
```

WellPaidDuck objects, which clearly are not meant to be employees, will work with the preceding loop. To prove this, let's create objects of our classes CommissionEmployee, SalariedCommissionEmployee and WellPaidDuck and place them in a list:

[lick here to view code image](#)

```
In [2]: from decimal import Decimal

In [3]: from commissionemployee import CommissionEmployee

In [4]: from salariedcommissionemployee import SalariedCommissionEmployee

In [5]: c = CommissionEmployee('Sue', 'Jones', '333-33-3333',
...:                             Decimal('10000.00'), Decimal('0.06'))
...:

In [6]: s = SalariedCommissionEmployee('Bob', 'Lewis', '444-44-4444',
...:                                     Decimal('5000.00'), Decimal('0.04'), Decimal('300.00'))
...:

In [7]: d = WellPaidDuck()

In [8]: employees = [c, s, d]
```

Now, let's process the list using the loop from section 10.8.3. As you can see in the output, Python is able to use duck typing to *polymorphically* process all three objects in the list:

[lick here to view code image](#)

```
In [9]: for employee in employees:
...:     print(employee)
...:     print(f'{employee.earnings():.2f}\n')
...:

CommissionEmployee: Sue Jones
social security number: 333-33-3333
gross sales: 10000.00
commission rate: 0.06
600.00

SalariedCommissionEmployee: Bob Lewis
social security number: 444-44-4444
gross sales: 5000.00
commission rate: 0.04
base salary: 300.00
```

```
500.00
```

```
I am a well-paid duck  
1,000,000.00
```

10.10 OPERATOR OVERLOADING

You’ve seen that you can interact with objects by accessing their attributes and properties and by calling their methods. Method-call notation can be cumbersome for certain kinds of operations, such as arithmetic. In these cases, it would be more convenient to use Python’s rich set of built-in operators.

This section shows how to use **operator overloading** to define how Python’s operators should handle objects of your own types. You’ve already used operator overloading frequently across wide ranges of types. For example, you’ve used:

- the `+` operator for adding numeric values, concatenating lists, concatenating strings and adding a value to every element in a NumPy array.
- the `[]` operator for accessing elements in lists, tuples, strings and arrays and for accessing the value for a specific key in a dictionary.
- the `*` operator for multiplying numeric values, repeating a sequence and multiplying every element in a NumPy array by a specific value.

You can overload most operators. For every overloadable operator, class `object` defines a special method, such as `__add__` for the addition (`+`) operator or `__mul__` for the multiplication (`*`) operator. Overriding these methods enables you to define how a given operator works for objects of your custom class. For a complete list of special methods, see

```
https://docs.python.org/3/reference/datamodel.html#special-method-names
```

Operator Overloading Restrictions

There are some restrictions on operator overloading:

- The precedence of an operator cannot be changed by overloading. However, parentheses can be used to force evaluation order in an expression.
- The left-to-right or right-to-left grouping of an operator cannot be changed by overloading.
- The “arity” of an operator—that is, whether it’s a unary or binary operator—cannot be changed.
- You cannot create new operators—only existing operators can be overloaded.

- The meaning of how an operator works on objects of built-in types cannot be changed. You cannot, for example, change `+` so that it subtracts two integers.
- Operator overloading works only with objects of custom classes or with a mixture of an object of a custom class and an object of a built-in type.

Complex Numbers

To demonstrate operator overloading, we'll define a class named `Complex` that represents complex numbers.⁸ Complex numbers, like $-3 + 4i$ and $6.2 - 11.73i$, have the form

⁸ Python has built-in support for complex values, so this class is simply for demonstration purposes.

```
realPart + imaginaryPart * i
```

where i is $\sqrt{-1}$. Like `ints`, `floats` and `Decimals`, complex numbers are arithmetic types. In this section, we'll create a class `Complex` that overloads just the `+` addition operator and the `+=` augmented assignment, so we can add `Complex` objects using Python's mathematical notations.

10.10.1 Test-Driving Class `Complex`

First, let's use class `Complex` to demonstrate its capabilities. We'll discuss the class's details in the next section. Import class `Complex` from `complexnumber.py`:

[lick here to view code image](#)

```
In [1]: from complexnumber import Complex
```

Next, create and display a couple of `Complex` objects. Snippets [3] and [5] implicitly call the `Complex` class's `__repr__` method to get a string representation of each object:

[lick here to view code image](#)

```
In [2]: x = Complex(real=2,    imaginary=4)

In [3]: x
Out[3]: (2 + 4i)

In [4]: y = Complex(real=5,    imaginary=-1)

In [5]: y
Out[5]: (5 - 1i)
```

We chose the `__repr__` string format shown in snippets [3] and [5] to mimic the `__repr__` strings produced by Python's built-in `complex` type.⁹

⁹ Python uses `j` rather than `i` for `.` For example, `3+4j` (with no spaces around the operator) creates a complex object with `real` and `imag` attributes. The `__repr__` string for this complex value is `' (3+4j) '`.

Now, let's use the `+` operator to add the `Complex` objects `x` and `y`. This expression adds the real parts of the two operands (2 and 5) and the imaginary parts of the two operands (`4i` and `-1i`), then returns a new `Complex` object containing the result:

```
In [6]: x + y
Out[6]: (7 + 3i)
```

The `+` operator does not modify either of its operands:

```
In [7]: x
Out[7]: (2 + 4i)

In [8]: y
Out[8]: (5 - 1i)
```

Finally, let's use the `+=` operator to add `y` to `x` and store the result in `x`. The `+=` operator *modifies* its left operand but not its right operand:

```
In [9]: x += y

In [10]: x
Out[10]: (7 + 3i)

In [11]: y
Out[11]: (5 - 1i)
```

10.10.2 Class `Complex` Definition

Now that we've seen class `Complex` in action, let's look at its definition to see how those capabilities were implemented.

Method `__init__`

The class's `__init__` method receives parameters to initialize the `real` and `imaginary` data attributes:

[lick here to view code image](#)

```
1 # complexnumber.py
2 """Complex class with overloaded operators."""
3
4 class Complex:
5     """Complex class that represents a complex number
6     with real and imaginary parts."""
7
```

```
8     def __init__(self, real, imaginary):
9         """Initialize Complex class's attributes."""
10        self.real = real
11        self.imaginary = imaginary
12
```

Overloaded + Operator

The following overridden special method `__add__` defines how to overload the `+` operator for use with two `Complex` objects:

[lick here to view code image](#)

```
13     def __add__(self, right):
14         """Overrides the + operator."""
15         return Complex(self.real + right.real,
16                        self.imaginary + right.imaginary)
17
```

Methods that overload binary operators must provide two parameters—the *first* (`self`) is the *left* operand and the *second* (`right`) is the *right* operand. Class `Complex`'s `__add__` method takes two `Complex` objects as arguments and returns a new `Complex` object containing the sum of the operands' `real` parts and the sum of the operands' `imaginary` parts.

We do *not* modify the contents of either of the original operands. This matches our intuitive sense of how this operator should behave. Adding two numbers does not modify either of the original values.

Overloaded += Augmented Assignment

Lines 18–22 overload special method `__iadd__` to define how the `+=` operator adds two `Complex` objects:

[lick here to view code image](#)

```
18     def __iadd__(self, right):
19         """Overrides the += operator."""
20         self.real += right.real
21         self.imaginary += right.imaginary
22         return self
23
```

Augmented assignments modify their left operands, so method `__iadd__` modifies the `self` object, which represents the left operand, then returns `self`.

Method `__repr__`

Lines 24–28 return the string representation of a `Complex` number.

[lick here to view code image](#)

```

24     def __repr__(self):
25         """Return string representation for repr()."""
26         return (f'({self.real} ' +
27                 ('+' if self.imaginary >= 0 else '-') +
28                 f' {abs(self.imaginary)}i)')

```

10.11 EXCEPTION CLASS HIERARCHY AND CUSTOM EXCEPTIONS

In the previous chapter, we introduced exception handling. Every exception is an object of a class in Python's exception class hierarchy⁰ or an object of a class that inherits from one of those classes. Exception classes inherit directly or indirectly from base class

`BaseException` and are defined in module `exceptions`.

⁰ <https://docs.python.org/3/library/exceptions.html>.

Python defines four primary `BaseException` subclasses—`SystemExit`, `KeyboardInterrupt`, `GeneratorExit` and `Exception`:

- `SystemExit` terminates program execution (or terminates an interactive session) and when uncaught does not produce a traceback like other exception types.
- `KeyboardInterrupt` exceptions occur when the user types the interrupt command—`Ctrl + C` (or *control + C*) on most systems.
- `GeneratorExit` exceptions occur when a generator closes—normally when a generator finishes producing values or when its `close` method is called explicitly.
- `Exception` is the base class for most common exceptions you'll encounter. You've seen exceptions of the `Exception` subclasses `ZeroDivisionError`, `NameError`, `ValueError`, `StatisticsError`, `TypeError`, `IndexError`, `KeyError`, `RuntimeError` and `AttributeError`. Often, `StandardErrors` can be caught and handled, so the program can continue running.

Catching Base-Class Exceptions

One of the benefits of the exception class hierarchy is that an `except` handler can catch exceptions of a particular type or can use a base-class type to catch those base-class exceptions and all related subclass exceptions. For example, an `except` handler that specifies the base class `Exception` can catch objects of *any* subclass of `Exception`. Placing an `except` handler that catches type `Exception` before other `except` handlers is a logic error, because all exceptions would be caught before other exception handlers could be reached. Thus, subsequent exception handlers are unreachable.

Custom Exception Classes

When you raise an exception from your code, you should generally use one of the existing

exception classes from the Python Standard Library. However, using the inheritance techniques presented earlier in this chapter, you can create your own custom exception classes that derive directly or indirectly from class `Exception`. Generally, that's discouraged, especially among novice programmers. Before creating custom exception classes, look for an appropriate existing exception class in the Python exception hierarchy. Define new exception classes only if you need to catch and handle the exceptions differently from other existing exception types. That should be rare.

10.12 NAMED TUPLES

You've used tuples to aggregate several data attributes into a single object. The Python Standard Library's **`collections` module** also provides **`named tuples`** that enable you to reference a tuple's members by name rather than by index number.

Let's create a simple named tuple that might be used to represent a card in a deck of cards. First, import function `namedtuple`:

[lick here to view code image](#)

```
In [1]: from collections import namedtuple
```

Function **`namedtuple`** creates a subclass of the built-in tuple type. The function's first argument is your new type's name and the second is a list of strings representing the identifiers you'll use to reference the new type's members:

[lick here to view code image](#)

```
In [2]: Card = namedtuple('Card', ['face', 'suit'])
```

We now have a new tuple type named `Card` that we can use anywhere a tuple can be used. Let's create a `Card` object, access its members by name and display its string representation:

[lick here to view code image](#)

```
In [3]: card = Card(face='Ace', suit='Spades')

In [4]: card.face
Out[4]: 'Ace'

In [5]: card.suit
Out[5]: 'Spades'

In [6]: card
Out[6]: Card(face='Ace', suit='Spades')
```

Other Named Tuple Features

Each named tuple type has additional methods. The type's `_make class method` (that is, a method called on the *class*) receives an iterable of values and returns an object of the named tuple type:

[lick here to view code image](#)

```
In [7]: values = ['Queen', 'Hearts']

In [8]: card = Card._make(values)

In [9]: card
Out[9]: Card(face='Queen', suit='Hearts')
```

This could be useful, for example, if you have a named tuple type representing records in a CSV file. As you read and tokenize CSV records, you could convert them into named tuple objects.

For a given object of a named tuple type, you can get an `OrderedDict` dictionary representation of the object's member names and values. An `OrderedDict` remembers the order in which its key–value pairs were inserted in the dictionary:

[lick here to view code image](#)

```
In [10]: card._asdict()
Out[10]: OrderedDict([('face', 'Queen'), ('suit', 'Hearts')])
```

For additional named tuple features see:

<https://docs.python.org/3/library/collections.html#collections.namedtuple>

10.13 A BRIEF INTRO TO PYTHON 3.7'S NEW DATA CLASSES

Though named tuples allow you to reference their members by name, they're still just tuples, not classes. For some of the benefits of named tuples, plus the capabilities that traditional Python classes provide, you can use Python 3.7's new **data classes**¹ from the Python Standard Library's **dataclasses module**.

¹ <https://www.python.org/dev/peps/pep-0557/>.

Data classes are among Python 3.7's most important new features. They help you build classes *faster* by using more *concise* notation and by *autogenerating* “boilerplate” code that's common in most classes. They could become the preferred way to define many Python classes. In this section, we'll present data-class fundamentals. At the end of the section, we'll provide links to more information.

Data Classes Autogenerate Code

Most classes you'll define provide an `__init__` method to create and initialize an object's attributes and a `__repr__` method to specify an object's custom string representation. If a class has many data attributes, creating these methods can be tedious.

Data classes *autogenerate* the data attributes and the `__init__` and `__repr__` methods for you. This can be particularly useful for classes that primarily aggregate related data items. For example, in an application that processes CSV records, you might want a class that represents each record's fields as data attributes in an object. Data classes also can be generated *dynamically* from a list of field names.

Data classes also autogenerate method `__eq__`, which overloads the `==` operator. Any class that has an `__eq__` method also implicitly supports `!=`. *All* classes inherit class object's default `__ne__` (not equals) method implementation, which returns the opposite of `__eq__` (or `NotImplemented` if the class does not define `__eq__`). Data classes do *not* automatically generate methods for the `<`, `<=`, `>` and `>=` comparison operators, but they can.

10.13.1 Creating a Card Data Class

Let's reimplement class `Card` from section 10.6.2 as a data class. The new class is defined in `carddataclass.py`. As you'll see, defining a data class requires some new syntax. In the subsequent subsections, we'll use our new `Card` data class in class `DeckOfCards` to show that it's interchangeable with the original `Card` class, then discuss some of the benefits of data classes over named tuples and traditional Python classes.

Importing from the `dataclasses` and `typing` Modules

The Python Standard Library's `dataclasses` module defines decorators and functions for implementing data classes. We'll use the **@dataclass decorator** (imported at line 4) to specify that a new class is a data class and causes various code to be written for you. Recall that our original `Card` class defined *class variables* `FACES` and `SUITS`, which are lists of the strings used to initialize `Cards`. We use **ClassVar** and **List** from the Python Standard Library's **typing module** (imported at line 5) to indicate that `FACES` and `SUITS` are *class variables* that refer to *lists*. We'll say more about these momentarily:

[lick here to view code image](#)

```
1 # carddataclass.py
2 """Card data class with class attributes, data attributes,
3 autogenerated methods and explicitly defined methods."""
4 from dataclasses import dataclass
5 from typing import ClassVar, List
6
```

Using the `@dataclass` Decorator

To specify that a class is a *data class*, precede its definition with the `@dataclass`

decorator: ²

² <https://docs.python.org/3/library/dataclasses.html#module-level-decorators-classes--and-functions>.

```
7 @dataclass
8 class Card:
```

Optionally, the `@dataclass` decorator may specify parentheses containing arguments that help the data class determine what autogenerated methods to include. For example, the decorator `@dataclass(order=True)` would cause the data class to autogenerated overloaded comparison operator methods for `<`, `<=`, `>` and `>=`. This might be useful, for example, if you need to sort your data-class objects.

Variable Annotations: Class Attributes

Unlike regular classes, data classes declare both class attributes and data attributes *inside* the class, but *outside* the class's methods. In a regular class, only *class attributes* are declared this way, and data attributes typically are created in `__init__`. Data classes require additional information, or *hints*, to distinguish class attributes from data attributes, which also affects the autogenerated methods' implementation details.

Lines 9–11 define and initialize the *class attributes* `FACES` and `SUITS`:

[lick here to view code image](#)

```
9     FACES: ClassVar[List[str]] = ['Ace', '2', '3', '4', '5', '6', '7',
10                                '8', '9', '10', 'Jack', 'Queen', 'King']
11     SUITS: ClassVar[List[str]] = ['Hearts', 'Diamonds', 'Clubs', 'Spades']
12
```

In lines 9 and 11, The notation

```
: ClassVar[List[str]]
```

is a **variable annotation** ^{3, 4} (sometimes called a *type hint*) specifying that `FACES` is a class attribute (`ClassVar`) which refers to a *list* of strings (`List[str]`). `SUITS` also is a class attribute which refers to a list of strings.

³ <https://www.python.org/dev/peps/pep-0526/>.

⁴Variable annotations are a recent language feature and are optional for regular classes. You will not see them in most legacy Python code.

Class variables are initialized in their definitions and are specific to the *class*, not individual *objects* of the class. Methods `__init__`, `__repr__` and `__eq__`, however, are for use with

objects of the class. When a data class generates these methods, it inspects all the variable annotations and includes only the *data attributes* in the method implementations.

Variable Annotations: Data Attributes

Normally, we create an object's data attributes in the class's `__init__` method (or methods called by `__init__`) via assignments of the form `self.attribute_name = value`. Because a data class *autogenerates* its `__init__` method, we need another way to specify data attributes in a data class's definition. We cannot simply place their names inside the class, which generates a `NameError`, as in:

[lick here to view code image](#)

```
In [1]: from dataclasses import dataclass

In [2]: @dataclass
...: class Demo:
...:     x # attempting to create a data attribute x
...:
-----
NameError                                Traceback (most recent call last)
<ipython-input-2-79ffe37b1ba2> in <module>()
----> 1 @dataclass
      2 class Demo:
      3     x # attempting to create a data attribute x
      4

<ipython-input-2-79ffe37b1ba2> in Demo()
      1 @dataclass
      2 class Demo:
----> 3     x # attempting to create a data attribute x
      4

NameError: name 'x' is not defined
```

Like class attributes, each data attribute must be declared with a variable annotation. Lines 13–14 define the data attributes `face` and `suit`. The variable annotation `: str` indicates that each should refer to string objects:

```
13     face: str
14     suit: str
```

Defining a Property and Other Methods

Data classes are classes, so they may contain properties and methods and participate in class hierarchies. For this `Card` data class, we defined the same read-only `image_name` property and custom special methods `__str__` and `__format__` as in our original `Card` class earlier in the chapter:

[lick here to view code image](#)

```

15     @property
16     def image_name(self):
17         """Return the Card's image file name."""
18         return str(self).replace(' ', '_') + '.png'
19
20     def __str__(self):
21         """Return string representation for str()."""
22         return f'{self.face} of {self.suit}'
23
24     def __format__(self, format):
25         """Return formatted string representation."""
26         return f'{str(self):{format}}'

```

Variable Annotation Notes

You can specify variable annotations using built-in type names (like `str`, `int` and `float`), class types or types defined by the `typing` module (such as `ClassVar` and `List` shown earlier). Even with type annotations, Python is still a *dynamically typed language*. So, type annotations are *not* enforced at execution time. So, even though a Card's face is meant to be a string, you can assign any type of object to `face`.

10.13.2 Using the Card Data Class

Let's demonstrate the new Card data class. First, create a Card:

[lick here to view code image](#)

```

In [1]: from carddataclass import Card

In [2]: c1 = Card(Card.FACES[0], Card.SUITS[3])

```

Next, let's use Card's autogenerated `__repr__` method to display the Card:

[lick here to view code image](#)

```

In [3]: c1
Out[3]: Card(face='Ace', suit='Spades')

```

Our custom `__str__` method, which `print` calls when passing it a Card object, returns a string of the form `'face of suit'`:

```

In [4]: print(c1)
Ace of Spades

```

Let's access our data class's attributes and read-only property:

[lick here to view code image](#)

```

In [5]: c1.face

```

```
Out[5]: 'Ace'

In [6]: c1.suit
Out[6]: 'Spades'

In [7]: c1.image_name
Out[7]: 'Ace_of_Spades.png'
```

Next, let's demonstrate that `Card` objects can be compared via the *autogenerated* `==` operator and inherited `!=` operator. First, create two additional `Card` objects—one identical to the first and one different:

[lick here to view code image](#)

```
In [8]: c2 = Card(Card.FACES[0], Card.SUITS[3])

In [9]: c2
Out[9]: Card(face='Ace', suit='Spades')

In [10]: c3 = Card(Card.FACES[0], Card.SUITS[0])

In [11]: c3
Out[11]: Card(face='Ace', suit='Hearts')
```

Now, compare the objects using `==` and `!=`:

```
In [12]: c1 == c2
Out[12]: True

In [13]: c1 == c3
Out[13]: False

In [14]: c1 != c3
Out[14]: True
```

Our `Card` data class is interchangeable with the `Card` class developed earlier in this chapter. To demonstrate this, we created the `deck2.py` file containing a copy of class `DeckOfCards` from earlier in the chapter and imported the `Card` data class into the file. The following snippets import class `DeckOfCards`, create an object of the class and print it. Recall that `print` implicitly calls the `DeckOfCards` `__str__` method, which formats each `Card` in a field of 19 characters, resulting in a call to each `Card`'s `__format__` method. Read each row left-to-right to confirm that all the `Cards` are displayed in order from each suit (Hearts, Diamonds, Clubs and Spades):

[lick here to view code image](#)

```
In [15]: from deck2 import DeckOfCards # uses Card data class

In [16]: deck_of_cards = DeckOfCards()

In [17]: print(deck_of_cards)
```

Ace of Hearts	2 of Hearts	3 of Hearts	4 of Hearts
5 of Hearts	6 of Hearts	7 of Hearts	8 of Hearts
9 of Hearts	10 of Hearts	Jack of Hearts	Queen of Hearts
King of Hearts	Ace of Diamonds	2 of Diamonds	3 of Diamonds
4 of Diamonds	5 of Diamonds	6 of Diamonds	7 of Diamonds
8 of Diamonds	9 of Diamonds	10 of Diamonds	Jack of Diamonds
Queen of Diamonds	King of Diamonds	Ace of Clubs	2 of Clubs
3 of Clubs	4 of Clubs	5 of Clubs	6 of Clubs
7 of Clubs	8 of Clubs	9 of Clubs	10 of Clubs
Jack of Clubs	Queen of Clubs	King of Clubs	Ace of Spades
2 of Spades	3 of Spades	4 of Spades	5 of Spades
6 of Spades	7 of Spades	8 of Spades	9 of Spades
10 of Spades	Jack of Spades	Queen of Spades	King of Spades

10.13.3 Data Class Advantages over Named Tuples

Data classes offer several advantages over named tuples ⁵:

⁵ <https://www.python.org/dev/peps/pep-0526/>.

- Although each named tuple technically represents a different type, a named tuple *is* a tuple and *all* tuples can be compared to one another. So, objects of *different* named tuple types could compare as equal if they have the same number of members and the same values for those members. Comparing objects of different data classes *always* returns `False`, as does comparing a data class object to a tuple object.
- If you have code that unpacks a tuple, adding more members to that tuple breaks the unpacking code. Data class objects cannot be unpacked. So you can add more data attributes to a data class without breaking existing code.
- A data class can be a base class or a subclass in an inheritance hierarchy.

10.13.4 Data Class Advantages over Traditional Classes

Data classes also offer various advantages over the traditional Python classes you saw earlier in this chapter:

- A data class autogenerates `__init__`, `__repr__` and `__eq__`, saving you time.
- A data class can autogenerate the special methods that overload the `<`, `<=`, `>` and `>=` comparison operators.
- When you change data attributes defined in a data class, then use it in a script or interactive session, the autogenerated code updates automatically. So, you have less code to maintain and debug.
- The required variable annotations for class attributes and data attributes enable you to take advantage of static code analysis tools. So, you might be able to eliminate additional errors before they can occur at execution time.

- Some static code analysis tools and IDEs can inspect variable annotations and issue warnings if your code uses the wrong type. This can help you locate logic errors in your code *before* you execute it.

More Information

Data classes have additional capabilities, such as creating “frozen” instances which do not allow you to assign values to a data class object’s attributes after the object is created. For a complete list of data class benefits and capabilities, see

```
https://www.python.org/dev/peps/pep-0557/
```

and

```
https://docs.python.org/3/library/dataclasses.html
```

10.14 UNIT TESTING WITH DOCSTRINGS AND DOCTEST

A key aspect of software development is testing your code to ensure that it works correctly. Even with extensive testing, however, your code may still contain bugs. According to the famous Dutch computer scientist Edsger Dijkstra, “Testing shows the presence, not the absence of bugs.”⁶

⁶J. N. Buxton and B. Randell, eds, *Software Engineering Techniques*, April 1970, p. 16. Report on a conference sponsored by the NATO Science Committee, Rome, Italy, 2731 October 1969

Module `doctest` and the `testmod` Function

The Python Standard Library provides the **`doctest` module** to help you test your code and conveniently retest it after you make modifications. When you execute the `doctest` module’s **`testmod` function**, it inspects your functions’, methods’ and classes’ docstrings looking for sample Python statements preceded by `>>>`, each followed on the next line by the given statement’s expected output (if any).⁷ The `testmod` function then executes those statements and confirms that they produce the expected output. If they do not, `testmod` reports errors indicating which tests failed so you can locate and fix the problems in your code. Each test you define in a docstring typically tests a specific *unit of code*, such as a function, a method or a class. Such tests are called **unit tests**.

⁷The notation `>>>` mimics the standard `python` interpreters input prompts.

Modified `Account` Class

The file `accountdoctest.py` contains the class `Account` from this chapter’s first example. We modified the `__init__` method’s docstring to include four tests which can be used to

ensure that the method works correctly:

- The test in line 11 creates a sample `Account` object named `account1`. This statement does not produce any output.
- The test in line 12 shows what the value of `account1`'s `name` attribute should be if line 11 executed successfully. The sample output is shown in line 13.
- The test in line 14 shows what the value of `account1`'s `balance` attribute should be if line 11 executed successfully. The sample output is shown in line 15.
- The test in line 18 creates an `Account` object with an invalid initial balance. The sample output shows that a `ValueError` exception should occur in this case. For exceptions, the `doctest` module's documentation recommends showing just the first and last lines of the traceback.⁸

⁸ <https://docs.python.org/3/library/doctest.html?highlight=doctest#module-doctest>.

You can intersperse your tests with descriptive text, such as line 17.

[click here to view code image](#)

```
1 # accountdoctest.py
2 """Account class definition."""
3 from decimal import Decimal
4
5 class Account:
6     """Account class for demonstrating doctest."""
7
8     def __init__(self, name, balance):
9         """Initialize an Account object.
10
11         >>> account1 = Account('John Green', Decimal('50.00'))
12         >>> account1.name
13         'John Green'
14         >>> account1.balance
15         Decimal('50.00')
16
17         The balance argument must be greater than or equal to 0.
18         >>> account2 = Account('John Green', Decimal('-50.00'))
19         Traceback (most recent call last):
20             ...
21         ValueError: Initial balance must be >= to 0.00.
22         """
23
24         # if balance is less than 0.00, raise an exception
25         if balance < Decimal('0.00'):
26             raise ValueError('Initial balance must be >= to 0.00.')
27
28         self.name = name
29         self.balance = balance
30
```

```

31     def deposit(self, amount):
32         """Deposit money to the account."""
33
34         # if amount is less than 0.00, raise an exception
35         if amount < Decimal('0.00'):
36             raise ValueError('amount must be positive.')
37
38         self.balance += amount
39
40 if __name__ == '__main__':
41     import doctest
42     doctest.testmod(verbose=True)

```

Module `__main__`

When you load any module, Python assigns a string containing the module's name to a global attribute of the module called `__name__`. When you execute a Python source file (such as `accountdoctest.py`) as a *script*, Python uses the string `'__main__'` as the module's name. You can use `__name__` in an `if` statement like lines 40–42 to specify code that should execute only if the source file is executed as a *script*. In this example, line 41 imports the `doctest` module and line 42 calls the module's `testmod` function to execute the docstring unit tests.

Running Tests

Run the file `accountdoctest.py` as a script to execute the tests. By default, if you call `testmod` with no arguments, it does not show test results for *successful* tests. In that case, if you get no output, all the tests executed successfully. In this example, line 42 calls `testmod` with the keyword argument `verbose=True`. This tells `testmod` to produce verbose output showing *every* test's results:

[lick here to view code image](#)

```

Trying:
    account1 = Account('John Green', Decimal('50.00'))
Expecting nothing
ok
Trying:
    account1.name
Expecting:
    'John Green'
ok
Trying:
    account1.balance
Expecting:
    Decimal('50.00')
ok
Trying:
    account2 = Account('John Green', Decimal('-50.00'))
Expecting:
    Traceback (most recent call last):
        ...
    ValueError: Initial balance must be >= to 0.00.
ok

```

```

3 items had no tests:
  __main__
  __main__.Account
  __main__.Account.deposit
1 items passed all tests:
   4 tests in __main__.Account.__init__
4 tests in 4 items.
4 passed and 0 failed.
Test passed.

```

In verbose mode, `testmod` shows for each test what it's "Trying" to do and what it's "Expecting" as a result, followed by "ok" if the test is successful. After completing the tests in verbose mode, `testmod` shows a summary of the results.

To demonstrate a *failed* test, “comment out” lines 25–26 in `accountdoctest.py` by preceding each with a `#`, then run `accountdoctest.py` as a script. To save space, we show just the portions of the doctest output indicating the failed test:

[lick here to view code image](#)

```

...
*****
File "accountdoctest.py", line 18, in  __main__.Account.__init__
Failed example:
    account2 = Account('John Green', Decimal('-50.00'))
Expected:
    Traceback (most recent call last):
        ...
    ValueError: Initial balance must be >= to 0.00.
Got nothing
*****
1 items had failures:
   1 of   4 in __main__.Account.__init__
4 tests in 4 items.
3 passed and 1 failed.
***Test Failed*** 1 failures.

```

In this case, we see that line 18's test failed. The `testmod` function was *expecting* a traceback indicating that a `ValueError` was raised due to the invalid initial balance. That exception did *not* occur, so the test failed. As the programmer responsible for defining this class, this failing test would be an indication that something is wrong with the validation code in your `__init__` method.

IPython %doctest_mode Magic

A convenient way to create doctests for existing code is to use an IPython interactive session to test your code, then copy and paste that session into a docstring. IPython's `In []` and `Out []` prompts are not compatible with `doctest`, so IPython provides the magic `%doctest_mode` to display prompts in the correct `doctest` format. The magic toggles between the two prompt styles. The first time you execute `%doctest_mode`, IPython switches to `>>>` prompts for input and no output prompts. The second time you execute

`doctest_mode`, IPython switches back to `In []` and `Out []` prompts.

10.15 NAMESPACES AND SCOPES

In the “Functions” chapter, we showed that each identifier has a scope that determines where you can use it in your program, and we introduced the local and global scopes. Here we continue our discussion of scopes with an introduction to namespaces.

Scopes are determined by **namespaces**, which associate identifiers with objects and are implemented “under the hood” as dictionaries. All namespaces are independent of one another. So, the same identifier may appear in multiple namespaces. There are three primary namespaces—local, global and built-in.

Local Namespace

Each function and method has a **local namespace** that associates local identifiers (such as, parameters and local variables) with objects. The local namespace exists from the moment the function or method is called until it terminates and is accessible *only* to that function or method. In a function’s or method’s suite, *assigning* to a variable that does not exist creates a local variable and adds it to the local namespace. Identifiers in the local namespace are *in scope* from the point at which you define them until the function or method terminates.

Global Namespace

Each module has a **global namespace** that associates a module’s global identifiers (such as global variables, function names and class names) with objects. Python creates a module’s global namespace when it loads the module. A module’s global namespace exists and its identifiers are *in scope* to the code within that module until the program (or interactive session) terminates. An IPython session has its own global namespace for all the identifiers you create in that session.

Each module’s global namespace also has an identifier called `__name__` containing the module’s name, such as `'math'` for the `math` module or `'random'` for the `random` module. As you saw in the previous section’s `doctest` example, `__name__` contains `'__main__'` for a `.py` file that you run as a script.

Built-In Namespace

The **built-in namespace** contains associates identifiers for Python’s built-in functions (such as, `input` and `range`) and types (such as, `int`, `float` and `str`) with objects that define those functions and types. Python creates the built-in namespace when the interpreter starts executing. The built-in namespace’s identifiers remain *in scope* for all code until the program (or interactive session) terminates.⁹

⁹ This assumes you do not shadow the built-in functions or types by redefining their identifiers in a local or global namespace. We discussed shadowing in the Functions chapter.

Finding Identifiers in Namespaces

When you use an identifier, Python searches for that identifier in the currently accessible namespaces, proceeding from *local* to *global* to *built-in*. To help you understand the namespace search order, consider the following IPython session:

[lick here to view code image](#)

```
In [1]: z = 'global z'

In [2]: def print_variables():
...:     y = 'local y in print_variables'
...:     print(y)
...:     print(z)
...:

In [3]: print_variables()
local y in print_variables
global z
```

The identifiers you define in an IPython session are placed in the session's *global* namespace. When snippet [3] calls `print_variables`, Python searches the *local*, *global* and *built-in* namespaces as follows:

- Snippet [3] is not in a function or method, so the session's *global* namespace and the *built-in* namespace are currently accessible. Python first searches the session's *global* namespace, which contains `print_variables`. So `print_variables` is *in scope* and Python uses the corresponding object to call `print_variables`.
- As `print_variables` begins executing, Python creates the function's *local* namespace. When function `print_variables` defines the local variable `y`, Python adds `y` to the function's *local* namespace. The variable `y` is now *in scope* until the function finishes executing.
- Next, `print_variables` calls the *built-in* function `print`, passing `y` as the argument. To execute this call, Python must resolve the identifiers `y` and `print`. The identifier `y` is defined in the *local* namespace, so it's *in scope* and Python will use the corresponding object (the string `'local y in print_variables'`) as `print`'s argument. To call the function, Python must find `print`'s corresponding object. First, it looks in the *local* namespace, which does *not* define `print`. Next, it looks in the session's *global* namespace, which does *not* define `print`. Finally, it looks in the *built-in* namespace, which *does* define `print`. So, `print` is *in scope* and Python uses the corresponding object to call `print`.
- Next, `print_variables` calls the *built-in* function `print` again with the argument `z`, which is *not* defined in the *local* namespace. So, Python looks in the *global* namespace. The argument `z` is defined in the *global* namespace, so `z` is *in scope* and Python will use the corresponding object (the string `'global z'`) as `print`'s argument. Again, Python

finds the identifier `print` in the *built-in* namespace and uses the corresponding object to call `print`.

- At this point, we reach the end of the `print_variables` function's suite, so the function terminates and its *local* namespace no longer exists, meaning the local variable `y` is now undefined.

To prove that `y` is undefined, let's try to display `y`:

[lick here to view code image](#)

```
In [4]: y
-----
NameError                                Traceback (most recent call last)
<ipython-input-4-9063a9f0e032> in <module>()
----> 1 y

NameError: name 'y' is not defined
```

In this case, there's no *local* namespace, so Python searches for `y` in the session's *global* namespace. The identifier `y` is *not* defined there, so Python searches for `y` in the *built-in* namespace. Again, Python does not find `y`. There are no more namespaces to search, so Python raises a `NameError`, indicating that `y` is not defined.

The identifiers `print_variables` and `z` still exist in the session's *global* namespace, so we can continue using them. For example, let's evaluate `z` to see its value:

```
In [5]: z
Out[5]: 'global z'
```

Nested Functions

One namespace we did not cover in the preceding discussion is the **enclosing namespace**. Python allows you to define **nested functions** inside other functions or methods. For example, if a function or method performs the same task several times, you might define a nested function to avoid repeating code in the enclosing function. When you access an identifier inside a nested function, Python searches the nested function's *local* namespace first, then the *enclosing* function's namespace, then the *global* namespace and finally the *built-in* namespace. This is sometimes referred to as the **LEGB (local, enclosing, global, built-in) rule**.

Class Namespace

A class has a namespace in which its class attributes are stored. When you access a class attribute, Python looks for that attribute first in the class's namespace, then in the base class's namespace, and so on, until either it finds the attribute or it reaches class `object`. If the attribute is not found, a `NameError` occurs.

Object Namespace

Each object has its own namespace containing the object's methods and data attributes. The class's `__init__` method starts with an empty object (`self`) and adds each attribute to the object's namespace. Once you define an attribute in an object's namespace, clients using the object may access the attribute's value.

10.16 INTRO TO DATA SCIENCE: TIME SERIES AND SIMPLE LINEAR REGRESSION

We've looked at sequences, such as lists, tuples and arrays. In this section, we'll discuss **time series**, which are sequences of values (called **observations**) associated with points in time. Some examples are daily closing stock prices, hourly temperature readings, the changing positions of a plane in flight, annual crop yields and quarterly company profits. Perhaps the ultimate time series is the stream of time-stamped tweets coming from Twitter users worldwide. In the "Data Mining Twitter" chapter, we'll study Twitter data in depth.

In this section, we'll use a technique called simple linear regression to make predictions from time series data. We'll use the 1895 through 2018 January average high temperatures in New York City to predict future average January high temperatures and to estimate the average January high temperatures for years preceding 1895.

In the "Machine Learning" chapter, we'll revisit this example using the scikit-learn library. In the "Deep Learning" chapter, we'll use *recurrent neural networks (RNNs)* to analyze time series.

In later chapters, we'll see that time series are popular in financial applications and with the Internet of Things (IoT), which we'll discuss in the "Big Data: Hadoop, Spark, NoSQL and IoT" chapter.

In this section, we'll display graphs with Seaborn and pandas, which both use Matplotlib, so launch IPython with Matplotlib support:

```
ipython --matplotlib
```

Time Series

The data we'll use is a time series in which the observations are *ordered* by year. **Univariate time series** have *one* observation per time, such as the average of the January high temperatures in New York City for a particular year. **Multivariate time series** have *two or more* observations per time, such as temperature, humidity and barometric pressure readings in a weather application. Here, we'll analyze a univariate time series.

Two tasks often performed with time series are:

- **Time series analysis**, which looks at existing time series data for patterns, helping data analysts understand the data. A common analysis task is to look for **seasonality** in the

data. For example, in New York City, the monthly average high temperature varies significantly based on the seasons (winter, spring, summer or fall).

- **Time series forecasting**, which uses past data to predict the future.

We'll perform time series forecasting in this section.

Simple Linear Regression

Using a technique called **simple linear regression**, we'll make predictions by finding a linear relationship between the months (January of each year) and New York City's average January high temperatures. Given a collection of values representing an **independent variable** (the month/year combination) and a **dependent variable** (the average high temperature for that month/year), simple linear regression describes the relationship between these variables with a straight line, known as the **regression line**.

Linear Relationships

To understand the general concept of a linear relationship, consider Fahrenheit and Celsius temperatures. Given a Fahrenheit temperature, we can calculate the corresponding Celsius temperature using the following formula:

$$c = 5 / 9 * (f - 32)$$

In this formula, f (the Fahrenheit temperature) is the *independent variable*, and c (the Celsius temperature) is the *dependent variable*—each value of c *depends on* the value of f used in the calculation.

Plotting Fahrenheit temperatures and their corresponding Celsius temperatures produces a straight line. To show this, let's first create a `lambda` for the preceding formula and use it to calculate the Celsius equivalents of the Fahrenheit temperatures 0–100 in 10-degree increments. We store each Fahrenheit/Celsius pair as a tuple in `temps`:

[lick here to view code image](#)

```
In [1]: c = lambda f: 5 / 9 * (f - 32)

In [2]: temps = [(f, c(f)) for f in range(0, 101, 10)]
```

Next, let's place the data in a `DataFrame`, then use its **plot method** to display the linear relationship between the Fahrenheit and Celsius temperatures. The `plot` method's `style` keyword argument controls the data's appearance. The period in the string `'.-'` indicates that each point should appear as a dot, and the dash indicates that lines should connect the dots. We manually set the *y*-axis label to `'Celsius'` because the `plot` method shows `'Celsius'` only in the graph's upper-left corner legend, by default.

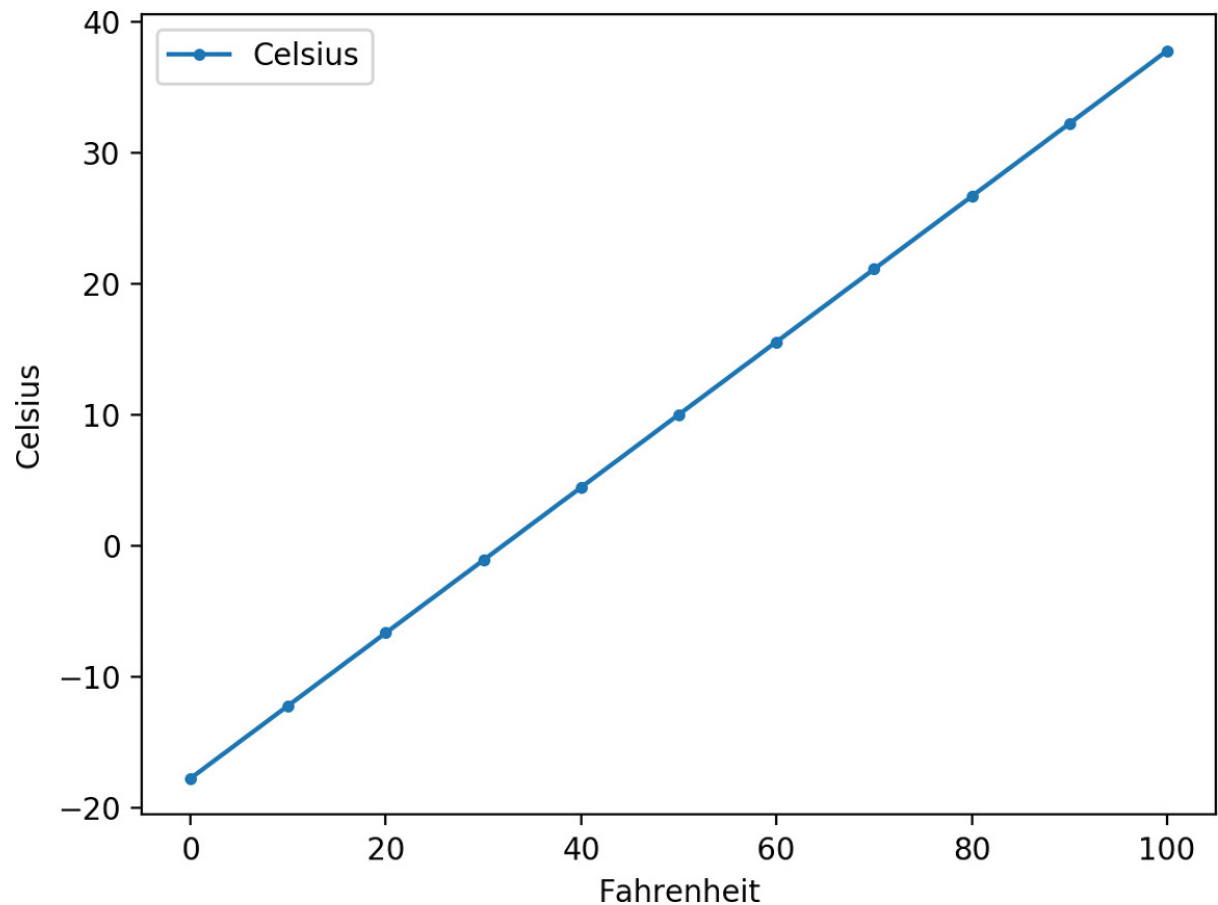
[lick here to view code image](#)

```
In [3]: import pandas as pd

In [4]: temps_df = pd.DataFrame(temps, columns=['Fahrenheit', 'Celsius'])

In [5]: axes = temps_df.plot(x='Fahrenheit', y='Celsius', style='.-')

In [6]: y_label = axes.set_ylabel('Celsius')
```



Components of the Simple Linear Regression Equation

The points along any straight line (in two dimensions) like those shown in the preceding graph can be calculated with the equation:

$$y = mx + b$$

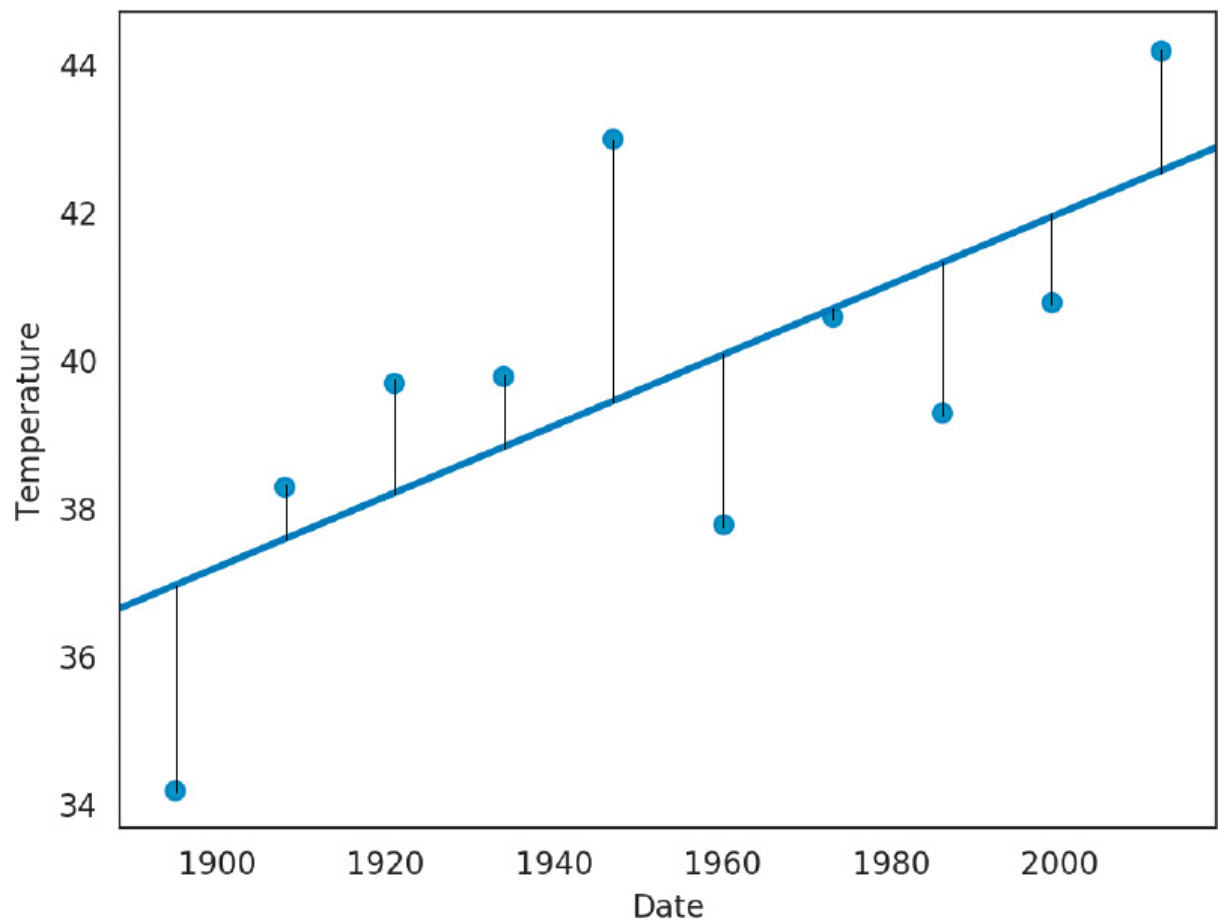
where

- m is the line's **slope**,
- b is the line's **intercept** with the y -axis (at $x = 0$),
- x is the independent variable (the date in this example), and
- y is the dependent variable (the temperature in this example).

In simple linear regression, y is the *predicted value* for a given x .

unction `linregress` from the SciPy's `stats` Module

Simple linear regression determines the slope (m) and intercept (b) of a straight line that best fits your data. Consider the following diagram, which shows a few of the time-series data points we'll process in this section and a corresponding regression line. We added vertical lines to indicate each data point's distance from the regression line:



The simple linear regression algorithm iteratively adjusts the slope and intercept and, for each adjustment, calculates the square of each point's distance from the line. The “best fit” occurs when the slope and intercept values minimize the sum of those squared distances. This is known as an **ordinary least squares** calculation.⁰

⁰ https://en.wikipedia.org/wiki/Ordinary_least_squares.

The **SciPy (Scientific Python) library** is widely used for engineering, science and math in Python. This library's `linregress` function (from the **`scipy.stats` module**) performs simple linear regression for you. After calling `linregress`, you'll plug the resulting slope and intercept into the $y = mx + b$ equation to make predictions.

Pandas

In the three previous Intro to Data Science sections, you used pandas to work with data. You'll continue using pandas throughout the rest of the book. In this example, we'll load the data for New York City's 1895–2018 average January high temperatures from a CSV file into a `DataFrame`. We'll then format the data for use in this example.

Seaborn Visualization

We'll use Seaborn to plot the `DataFrame`'s data with a regression line that shows the average high-temperature trend over the period 1895–2018.

Getting Weather Data from NOAA

Let's get the data for our study. The National Oceanic and Atmospheric Administration (NOAA) ¹ offers lots of public historical data including time series for average high temperatures in specific cities over various time intervals.

¹ <http://www.noaa.gov>.

We obtained the January average high temperatures for New York City from 1895 through 2018 from NOAA's "Climate at a Glance" time series at:

```
https://www.ncdc.noaa.gov/cag/
```

On that web page, you can select temperature, precipitation and other data for the entire U.S., regions within the U.S., states, cities and more. Once you've set the area and time frame, click **Plot** to display a diagram and view a table of the selected data. At the top of that table are links for downloading the data in several formats including CSV, which we discussed in the "Files and Exceptions" chapter. NOAA's maximum date range available at the time of this writing was 1895–2018. For your convenience, we provided the data in the `ch10` examples folder in the file `ave_hi_nyc_jan_1895-2018.csv`. If you download the data on your own, delete the rows above the line containing "Date, Value, Anomaly".

This data contains three columns per observation:

- `Date`—A value of the form 'YYYYMM' (such as '201801'). `MM` is always 01 because we downloaded data for only January of each year.
- `Value`—A floating-point Fahrenheit temperature.
- `Anomaly`—The difference between the value for the given date and average values for all dates. We do not use the `Anomaly` value in this example, so we'll ignore it.

Loading the Average High Temperatures into a `DataFrame`

Let's load and display the New York City data from `ave_hi_nyc_jan_1895-2018.csv`:

[lick here to view code image](#)

```
In [7]: nyc = pd.read_csv('ave_hi_nyc_jan_1895-2018.csv')
```

We can look at the `DataFrame`'s `head` and `tail` to get a sense of the data:

[lick here to view code image](#)